

Лекция 3: Ветвление (Условные операторы)

Введение

На прошлой лекции мы научились сравнивать числа и использовать логические операторы. Но просто уметь сравнивать недостаточно — нужно уметь **принимать решения** на основе этих сравнений. Именно для этого в программировании существуют **условные операторы**.

Представьте, что вы — программа. Вы идёте по улице и видите светофор. Ваше решение зависит от цвета:

- **Если** горит зелёный → идите
- **Иначе если** горит жёлтый → приготовьтесь
- **Иначе** (красный) → стойте

Сегодня мы изучим все способы организации ветвления в Java:

- **if-else** — для любых условий
- **Тернарный оператор** — краткая форма для простых условий
- **switch-case** — для множественного выбора по одному значению

1) Конструкция if-else

Конструкция if-else — это самый базовый и самый гибкий способ управления потоком выполнения программы. Она позволяет выполнять разные блоки кода в зависимости от условия.

Простой if

Самая простая форма — выполнить код, **если** условие истинно:

java

Копировать

Скачать

```
public class SimpleIf {  
    public static void main(String[] args) {  
        int age = 20;  
  
        if (age >= 18) {  
            System.out.println("Вы совершеннолетний.");  
            System.out.println("Можете получить права.");  
        }  
    }  
}
```

```
        System.out.println("Программа продолжает работу.");
    }
}
```

Важно: Если после `if` идёт только одна инструкция, фигурные скобки можно опустить, но **лучше всегда их ставить** — это делает код понятнее и защищает от ошибок:

```
java
Копировать
Скачать
// Так можно, но не рекомендуется
if (age >= 18)
    System.out.println("Вы совершеннолетний.");

// Так правильно
if (age >= 18) {
    System.out.println("Вы совершеннолетний.");
}
```

Конструкция `if-else`

Если нужно выполнить один код при истинном условии, и другой — при ложном:

```
java
Копировать
Скачать
public class IfElseExample {
    public static void main(String[] args) {
        int temperature = 25;

        if (temperature > 30) {
            System.out.println("Очень жарко! Включите кондиционер.");
        } else {
            System.out.println("Нормальная температура.");
        }

        // Проверка на чётность
        int number = 7;
        if (number % 2 == 0) {
            System.out.println(number + " — чётное число");
        } else {
            System.out.println(number + " — нечётное число");
        }
    }
}
```

```
    }  
  }  
}
```

Конструкция if-else if-else

Когда нужно проверить несколько условий последовательно:

```
public class IfElseIfExample {  
    public static void main(String[] args) {  
        int score = 85;  
  
        if (score >= 90) {  
            System.out.println("Отлично! Оценка 5");  
        } else if (score >= 75) {  
            System.out.println("Хорошо! Оценка 4");  
        } else if (score >= 60) {  
            System.out.println("Удовлетворительно. Оценка 3");  
        } else if (score >= 40) {  
            System.out.println("Неудовлетворительно. Оценка 2");  
        } else {  
            System.out.println("Плохо. Оценка 1");  
        }  
    }  
}
```

Как работает else if:

1. Проверяется первое условие (score >= 90). Если оно true — выполняется его блок, остальные игнорируются.
2. Если первое условие false — проверяется второе (score >= 75).
3. И так далее, пока не найдётся истинное условие.
4. Если ни одно не истинно — выполняется блок else.

Вложенные if

Условия можно вкладывать друг в друга:

```
public class NestedIf {  
    public static void main(String[] args) {  
        boolean hasTicket = true;  
        boolean hasPassport = true;  
        int age = 16;  
  
        if (hasTicket) {
```

```

    if (hasPassport) {
        if (age >= 18) {
            System.out.println("Добро пожаловать на борт!");
        } else {
            System.out.println("Вам нужен сопровождающий взрослый.");
        }
    } else {
        System.out.println("Нет паспорта — посадка запрещена.");
    }
} else {
    System.out.println("Купите билет сначала.");
}
}
}

```

Лучше избегать глубокой вложенности — это усложняет чтение кода. Часто вложенные if можно заменить логическими операторами:

// Вместо вложенных if

```

if (hasTicket && hasPassport && age >= 18) {
    System.out.println("Добро пожаловать на борт!");
} else if (hasTicket && hasPassport) {
    System.out.println("Вам нужен сопровождающий.");
} else if (hasTicket) {
    System.out.println("Нет паспорта.");
} else {
    System.out.println("Купите билет.");
}

```

Распространённая ошибка: точка с запятой после if

```
int x = 5;
```

// ОШИБКА! Точка с запятой завершает оператор if

```

if (x > 0); {
    System.out.println("x положительный");
}

```

// Этот println выполнится ВСЕГДА, независимо от условия!

// ПРАВИЛЬНО

```

if (x > 0) {
    System.out.println("x положительный");
}

```

2) Тернарный оператор

Тернарный оператор — это сокращённая форма записи простого if-else, который возвращает значение. Называется "тернарным", потому что у него три операнда:

условие ? значение_если_истина : значение_если_ложь

Базовый синтаксис

```
public class TernaryExample {
    public static void main(String[] args) {
        int age = 20;

        // Длинная запись через if-else
        String status;
        if (age >= 18) {
            status = "взрослый";
        } else {
            status = "ребёнок";
        }
        System.out.println("Статус: " + status);

        // Короткая запись через тернарный оператор
        String shortStatus = (age >= 18) ? "взрослый" : "ребёнок";
        System.out.println("Статус: " + shortStatus);

        // Тернарный оператор с числами
        int a = 10, b = 20;
        int max = (a > b) ? a : b;
        System.out.println("Максимум: " + max); // 20

        // Тернарный оператор внутри вывода
        System.out.println("Меньшее число: " + ((a < b) ? a : b));
    }
}
```

Где уместно использовать тернарный оператор

Тернарный оператор хорош для **простых** условий, где нужно выбрать одно из двух значений:

```
public class TernaryGoodPractices {
    public static void main(String[] args) {
        int number = 7;

        // Определение чётности
```

```

String parity = (number % 2 == 0) ? "чётное" : "нечётное";
System.out.println(number + " — " + parity);

// Проверка на положительность
int value = -5;
String sign = (value > 0) ? "положительное" : (value < 0) ? "отрицательное"
: "ноль";
System.out.println(value + " — " + sign);

// Минимум и максимум
int x = 15, y = 30;
int min = (x < y) ? x : y;
int max = (x > y) ? x : y;

// Скидка в зависимости от суммы
double total = 1500;
double discount = (total > 1000) ? total * 0.1 : total * 0.05;
System.out.println("Скидка: " + discount);
}
}

```

Где НЕ стоит использовать тернарный оператор

Не используйте тернарный оператор для сложной логики или когда нужно выполнить несколько действий:

// ПЛОХО: слишком сложно для понимания

```
int result = (a > b) ? (c > d ? (e > f ? a : b) : (g > h ? c : d)) : (i > j ? k : l);
```

// ПЛОХО: тернарный оператор для выполнения действий, а не возврата значения

```
(age >= 18) ? System.out.println("Взрослый") : System.out.println("Ребёнок");
```

// ХОРОШО: использовать if-else для действий

```
if (age >= 18) {
    System.out.println("Взрослый");
} else {
    System.out.println("Ребёнок");
}
```

Золотое правило: Используйте тернарный оператор только если он делает код **короче И понятнее**. Если сомневаетесь — используйте if-else.

3) Конструкция switch-case

switch-case используется, когда нужно выбрать один вариант из нескольких **фиксированных** значений одной переменной. Это альтернатива длинной цепочке else if.

Синтаксис switch-case

```
switch (выражение) {  
    case значение1:  
        // код для значения1  
        break;  
    case значение2:  
        // код для значения2  
        break;  
    case значение3:  
        // код для значения3  
        break;  
    default:  
        // код, если ни одно значение не подошло  
        break;  
}
```

Простой пример

```
public class SimpleSwitch {  
    public static void main(String[] args) {  
        int dayOfWeek = 3;  
  
        switch (dayOfWeek) {  
            case 1:  
                System.out.println("Понедельник");  
                break;  
            case 2:  
                System.out.println("Вторник");  
                break;  
            case 3:  
                System.out.println("Среда");  
                break;  
            case 4:  
                System.out.println("Четверг");  
                break;  
            case 5:  
                System.out.println("Пятница");  
                break;  
            case 6:  
                System.out.println("Суббота");  
                break;  
            case 7:  
                System.out.println("Воскресенье");  
                break;  
        }  
    }  
}
```

```

        System.out.println("Суббота");
        break;
    case 7:
        System.out.println("Воскресенье");
        break;
    default:
        System.out.println("Некорректный день недели");
        break;
    }
}
}

```

Ключевое слово break

break заставляет программу выйти из switch. Если забыть написать break, программа "провалится" в следующий case:

```

public class WithoutBreak {
    public static void main(String[] args) {
        int month = 2;

        switch (month) {
            case 1:
                System.out.print("Январь ");
            case 2:
                System.out.print("Февраль ");
            case 3:
                System.out.print("Март ");
            default:
                System.out.print("Другой месяц");
        }
        // Вывод: "Февраль Март Другой месяц"
    }
}

```

Иногда "проваливание" (fall-through) используют намеренно:

```

public class FallThroughExample {
    public static void main(String[] args) {
        int month = 7;
        int days;

        switch (month) {
            case 4:
            case 6:

```



```

        case 9:
        case 11:
            days = 30;
            break;
        case 2:
            days = 28;
            break;
        case 1:
        case 3:
        case 5:
        case 7:
        case 8:
        case 10:
        case 12:
            days = 31;
            break;
        default:
            days = 0;
            System.out.println("Неверный месяц");
            break;
    }

    if (days != 0) {
        System.out.println("В " + month + " месяце " + days + " дней");
    }
}

```

Ключевое слово default

Блок default выполняется, если значение не совпало ни с одним case. Его можно разместить в любом месте, но обычно ставят в конце:

```

char grade = 'B';

switch (grade) {
    case 'A':
        System.out.println("Отлично!");
        break;
    case 'B':
        System.out.println("Хорошо!");
        break;
    case 'C':
        System.out.println("Удовлетворительно.");
        break;
    default:
        System.out.println("Неизвестный рейтинг");
}

```

```

default:
    System.out.println("Неизвестная оценка.");
    break;
}

```

С чем работает switch

В классическом Java switch работает с:

Тип	Пример
int	case 1:
byte	case 10:
short	case 100:
char	case 'A':
String	case "Москва":
enum	case MONDAY:

Таблица 1. Работа Switch с разными типами данных

```

// Switch co String
public class SwitchString {
    public static void main(String[] args) {
        String city = "Москва";

        switch (city) {
            case "Москва":
                System.out.println("Столица России");
                break;
            case "Санкт-Петербург":
                System.out.println("Северная столица");
                break;
            case "Новосибирск":
                System.out.println("Столица Сибири");
                break;
            default:
                System.out.println("Город не найден");
                break;
        }
    }
}

```

```

// Switch c char
public class SwitchChar {
    public static void main(String[] args) {
        char operation = '+';
        int a = 10, b = 5;
    }
}

```

```

switch (operation) {
    case '+':
        System.out.println(a + " + " + b + " = " + (a + b));
        break;
    case '-':
        System.out.println(a + " - " + b + " = " + (a - b));
        break;
    case '*':
        System.out.println(a + " * " + b + " = " + (a * b));
        break;
    case '/':
        System.out.println(a + " / " + b + " = " + (a / b));
        break;
    default:
        System.out.println("Неизвестная операция");
        break;
}
}
}

```

Switch-выражения (Java 14+)

Начиная с Java 14, появился новый синтаксис switch как **выражения** (возвращает значение):

```

public class SwitchExpression {
    public static void main(String[] args) {
        int day = 3;

        // Стрелочный синтаксис (без break)
        String dayName = switch (day) {
            case 1 -> "Понедельник";
            case 2 -> "Вторник";
            case 3 -> "Среда";
            case 4 -> "Четверг";
            case 5 -> "Пятница";
            case 6 -> "Суббота";
            case 7 -> "Воскресенье";
            default -> "Неизвестный день";
        };
        System.out.println(dayName);

        // С несколькими значениями
        int month = 2;
    }
}

```

```

int days = switch (month) {
    case 1, 3, 5, 7, 8, 10, 12 -> 31;
    case 4, 6, 9, 11 -> 30;
    case 2 -> 28;
    default -> 0;
};
System.out.println("Дней: " + days);

// С блоком кода (нужен yield)
String season = switch (month) {
    case 12, 1, 2 -> {
        yield "Зима";
    }
    case 3, 4, 5 -> {
        yield "Весна";
    }
    case 6, 7, 8 -> {
        yield "Лето";
    }
    case 9, 10, 11 -> {
        yield "Осень";
    }
    default -> "Неизвестно";
};
System.out.println(season);
}
}

```

4) Сравнение разных способов ветвления

Когда что использовать?

Ситуация	Лучший выбор	Почему
Проверка сложного условия	if-else	Гибкость, можно использовать любые условия
Выбор из 2-3 вариантов	if-else или тернарный	Просто и понятно
Выбор из 4+ вариантов по одному значению	switch	Код становится чище и быстрее
Простое присваивание в зависимости от условия	Тернарный оператор	Кратко и выразительно
Выполнение действий в	if-else	Тернарный не предназначен

зависимости от условия		для действий
------------------------	--	--------------

Таблица 2. Использование ветвлений

Сравнение производительности

Для небольшого количества вариантов разница незначительна. Но теоретически:

- switch может быть быстрее для множества вариантов (использует таблицу переходов)
- if-else проверяет условия последовательно

// Этот код лучше через switch

```
if (x == 1) { ... }
else if (x == 2) { ... }
else if (x == 3) { ... }
```

// ... много вариантов

// switch будет более эффективен

```
switch(x) {
    case 1: ... break;
    case 2: ... break;
    case 3: ... break;
    // ...
}
```

5) Типичные ошибки

Ошибка 1: Забытый break в switch

// ОШИБКА

```
switch(day) {
    case 1:
        System.out.println("Понедельник");
    case 2:
        System.out.println("Вторник");
    // Без break выполнится и "Понедельник", и "Вторник"
}
```

// ПРАВИЛЬНО

```
switch(day) {
    case 1:
        System.out.println("Понедельник");
        break;
    case 2:
```

```
        System.out.println("Вторник");
        break;
    }
```

Ошибка 2: Путаница между = и == в условии

// ОШИБКА

```
if (x = 5) { // Ошибка компиляции
    System.out.println("x равен 5");
}
```

// ПРАВИЛЬНО

```
if (x == 5) {
    System.out.println("x равен 5");
}
```

Ошибка 3: Точка с запятой после if

// ОШИБКА

```
if (age >= 18); { // точка с запятой завершает if
    System.out.println("Вы совершеннолетний"); // Этот код выполнится всегд
a
}
```

// ПРАВИЛЬНО

```
if (age >= 18) {
    System.out.println("Вы совершеннолетний");
}
```

Ошибка 4: Неполная обработка всех вариантов

// ПЛОХО: нет обработки отрицательных чисел

```
int number = -5;
if (number > 0) {
    System.out.println("Положительное");
} else if (number == 0) {
    System.out.println("Ноль");
}
```

// Для отрицательных чисел ничего не выведется

// ХОРОШО: есть обработка всех случаев

```
if (number > 0) {
    System.out.println("Положительное");
} else if (number == 0) {
```

```
    System.out.println("Ноль");  
} else {  
    System.out.println("Отрицательное");  
}
```

Итоги третьей лекции

Сегодня мы изучили все основные способы организации ветвления в Java:

Конструкция if-else:

- if (условие) { ... } — выполнить, если условие истинно
- if (условие) { ... } else { ... } — выбор из двух вариантов
- if-else if-else — выбор из нескольких вариантов

Тернарный оператор:

- условие ? значение1 : значение2
- Краткая форма для простых if-else
- Используйте только когда код становится чище

Конструкция switch-case:

- Выбор из множества фиксированных значений
- Работает с int, char, String, enum
- break нужен для выхода
- default — если ничего не подошло

Ключевые выводы:

1. Ветвление — основа принятия решений в программах
2. Выбирайте инструмент в зависимости от задачи
3. if-else — самый гибкий, подходит для любых условий
4. switch — лучший выбор для множества фиксированных вариантов
5. Тернарный оператор — только для простых присваиваний